

Matalino AI Website - Production Deployment Checklist

Pre-Deployment Checklist

1. Environment Variables Setup

Required Production Environment Variables:

```
# Database
DATABASE_URL="postgresql://username:password@host:port/database"

# NextAuth.js Configuration
NEXTAUTH_URL="https://yourdomain.com"
NEXTAUTH_SECRET="your-super-secure-secret-key-32-chars-min"

# Stripe Configuration
STRIPE_SECRET_KEY="sk_live_your_live_stripe_secret_key"
STRIPE_PUBLISHABLE_KEY="pk_live_your_live_stripe_publishable_key"
STRIPE_WEBHOOK_SECRET="whsec_your_webhook_secret_from_stripe_dashboard"
STRIPE_PRICE_ID="price_your_live_price_id_from_stripe"
```

⚠ Critical Notes:

- Use `sk_live_` and `pk_live_` keys for production (not test keys)
- Generate NEXTAUTH_SECRET: `openssl rand -base64 32`
- NEXTAUTH_URL must match your production domain exactly
- Webhook secret comes from Stripe Dashboard after creating webhook endpoint

2. Database Setup

```
# Generate Prisma client
npx prisma generate

# Deploy migrations to production database
npx prisma migrate deploy

# Verify database connection
npx prisma db pull
```

3. Build Verification

```
# Clean install dependencies
yarn install --frozen-lockfile

# Build the application
yarn build

# Verify build success (should show route table)
```

4. Stripe Configuration

- [] Create webhook endpoint in Stripe Dashboard: `https://yourdomain.com/api/webhooks/stripe`
- [] Subscribe to events: `invoice.payment_succeeded`, `customer.subscription.updated`, `customer.subscription.deleted`
- [] Copy webhook signing secret to environment variables
- [] Test webhook with Stripe CLI: `stripe listen --forward-to https://yourdomain.com/api/webhooks/stripe`

Deployment Commands

For Vercel Deployment:

```
# Install Vercel CLI
npm i -g vercel

# Deploy to production
vercel --prod

# Set environment variables
vercel env add NEXTAUTH_URL production
vercel env add NEXTAUTH_SECRET production
vercel env add DATABASE_URL production
vercel env add STRIPE_SECRET_KEY production
vercel env add STRIPE_PUBLISHABLE_KEY production
vercel env add STRIPE_WEBHOOK_SECRET production
vercel env add STRIPE_PRICE_ID production
```

For Manual Server Deployment:

```
# Build the application
yarn build

# Start production server
yarn start

# Or with PM2 for process management
pm2 start yarn --name "matalino-ai" -- start
```

Post-Deployment Verification

1. Basic Functionality Tests

- [] Homepage loads correctly
- [] Pricing page displays both Free and Pro tiers
- [] Sign-in/Sign-up pages are accessible
- [] Dashboard redirects to sign-in when not authenticated

2. Authentication Tests

- [] User registration works
- [] User login works
- [] Session persistence works

- ☐ Protected routes redirect properly

3. Payment Integration Tests

- ☐ Stripe checkout button works
- ☐ Payment processing completes
- ☐ Webhook receives events
- ☐ User subscription status updates

4. Database Tests

- ☐ User data persists correctly
- ☐ Subscription data syncs with Stripe
- ☐ Usage tracking works

Troubleshooting Guide

Issue: “Latest changes not appearing”

Possible Causes & Solutions:

1. Build Cache Issues

```
bash
# Clear Next.js cache
rm -rf .next
yarn build
```

2. Environment Variables Not Set

```
bash
# Verify environment variables are loaded
node -e "console.log(process.env.NEXTAUTH_URL)"
```

3. Database Migration Not Applied

```
``bash
# Check migration status
npx prisma migrate status

# Deploy pending migrations
npx prisma migrate deploy
``
```

1. Deployment Platform Cache

- **Vercel:** Trigger new deployment or clear cache
- **Netlify:** Clear cache and redeploy
- **Manual:** Restart server process

Issue: “Authentication not working”

Solutions:

1. Verify NEXTAUTH_URL matches production domain exactly
2. Ensure NEXTAUTH_SECRET is set and secure
3. Check database connection and User table exists
4. Verify cookies are being set (check browser dev tools)

Issue: “Stripe payments failing”

Solutions:

1. Verify using live Stripe keys (not test keys)
2. Check webhook endpoint is accessible
3. Verify webhook secret matches Stripe dashboard
4. Test webhook with: `curl -X POST https://yourdomain.com/api/webhooks/stripe`

Issue: “Database connection errors”

Solutions:

1. Verify DATABASE_URL format and credentials
2. Check database server is accessible from deployment environment
3. Ensure database exists and migrations are applied
4. Test connection: `npx prisma db pull`

Issue: “Build failures”

Solutions:

1. Check for TypeScript errors: `yarn type-check`
2. Verify all dependencies are installed: `yarn install`
3. Check for missing environment variables during build
4. Review build logs for specific error messages

Production Monitoring

Health Checks

- ☐ Set up uptime monitoring for main endpoints
- ☐ Monitor database connection health
- ☐ Track Stripe webhook delivery success
- ☐ Monitor application performance metrics

Logging

- ☐ Configure error logging (Sentry, LogRocket, etc.)
- ☐ Monitor authentication failures
- ☐ Track payment processing errors
- ☐ Log webhook processing issues

Rollback Procedure

If deployment fails:

1. Immediate Rollback

```
``bash
# Vercel
vercel rollback
```

Manual deployment

`git checkout previous-working-commit`

yarn build && yarn start

...

1. Database Rollback (if needed)

```
bash
# Revert to previous migration
npx prisma migrate reset
npx prisma migrate deploy
```

2. Environment Variables

- Restore previous environment variable values
- Verify all services are working

Support Contacts

- **Database Issues:** Check hosting provider documentation
 - **Stripe Issues:** Stripe Support or documentation
 - **NextAuth Issues:** NextAuth.js documentation
 - **Deployment Platform:** Platform-specific support
-

Final Deployment Verification

After completing deployment:

1. Functional Testing

- [] Complete user registration flow
- [] Test payment processing end-to-end
- [] Verify dashboard functionality
- [] Test usage tracking

2. Performance Testing

- [] Page load times acceptable
- [] Database queries optimized
- [] No memory leaks

3. Security Testing

- [] HTTPS enforced
- [] Secure headers configured
- [] Environment variables secured
- [] No sensitive data exposed

Deployment Complete!

Your Matalino AI website with freemium SaaS features is now live in production.